torch.set_float32_matmul_precision#

https://pytorch.org/docs/stable/generated/torch.set_float32_matmul_precisior

Description:

Sets the internal precision of float32 matrix multiplications. Running float32 matrix multiplications in lower precision may significantly increase performance, and in some programs the loss of precision has a negligible impact. Supports three settings: “highest”, float32 matrix multiplications use the float32 datatype (24 mantissa bits with 23 bits explicitly stored) for internal computations. “high”, float32 matrix multiplications either use the TensorFloat32 datatype (10 mantissa bits explicitly stored) or treat each float32 number as the sum of two bfloat16 numbers (approximately 16 mantissa bits with 14 bits explicitly stored), if the appropriate fast matrix multiplication algorithms are available. Otherwise float32 matrix multiplications are computed as if the precision is “highest”. See below for more information on the bfloat16 approach.

Function Signature

```
torch.set_float32_matmul_precision(precision)[source]#
Henry2019
Parameters
```

Notes & Warnings

NOTE Note This does not change the output dtype of float32 matrix multiplications, it controls how the internal computation of the matrix multiplication is performed.

NOTE Note This does not change the precision of convolution operations. Other flags, like torch.backends.cudnn.allow_tf32, may control the precision of convolution operations.

NOTE Note This flag currently only affects one native device type: CUDA. If “high” or “medium” are set then the TensorFloat32 datatype will be used when computing float32 matrix multiplications, equivalent to setting torch.backends.cuda.matmul.allow_tf32 = True. When “highest” (the default) is set then the float32 datatype is used for internal computations, equivalent to setting torch.backends.cuda.matmul.allow_tf32 = False.

Detailed Documentation

Documentation

Sets the internal precision of float32 matrix multiplications.

Running float32 matrix multiplications in lower precision may significantly increase performance, and in some programs the loss of precision has a negligible impact.

Supports three settings:

“highest”, float32 matrix multiplications use the float32 datatype (24 mantissa bits with 23 bits explicitly stored) for internal computations.

“high”, float32 matrix multiplications either use the TensorFloat32 datatype (10 mantissa bits explicitly stored) or treat each float32 number as the sum of two bfloat16 numbers (approximately 16 mantissa bits with 14 bits explicitly stored), if the appropriate fast matrix multiplication algorithms are available. Otherwise float32 matrix multiplications are computed as if the precision is “highest”. See below for more information on the bfloat16 approach.

“medium”, float32 matrix multiplications use the bfloat16 datatype (8 mantissa bits with 7 bits explicitly stored) for internal computations, if a fast matrix multiplication algorithm using that datatype internally is available. Otherwise float32 matrix multiplications are computed as if the precision is “high”.

When using “high” precision, float32 multiplications may use a bfloat16-based algorithm that is more complicated than simply truncating to some smaller number mantissa bits (e.g. 10 for TensorFloat32, 7 for bfloat16 explicitly stored). Refer to [Henry2019] for a complete description of this algorithm. To briefly explain here, the first step is to realize that we can perfectly encode a single float32 number as the sum of three bfloat16 numbers (because float32 has 23 mantissa bits while bfloat16 has 7 explicitly stored, and both have the same number of exponent bits). This means that the product of two float32 numbers can be exactly given by the sum of nine products of bfloat16 numbers. We can then trade accuracy for speed by dropping some of these products. The “high” precision algorithm specifically keeps only the three most significant products, which conveniently excludes all of the products involving the last 8

mantissa bits of either input. This means that we can represent our inputs as the sum of two bfloat16 numbers rather than three. Because bfloat16 fused-multiply-add (FMA) instructions are typically >10x faster than float32 ones, it's faster to do three multiplications and 2 additions with bfloat16 precision than it is to do a single multiplication with float32 precision.

http://arxiv.org/abs/1904.06376

This does not change the output dtype of float32 matrix multiplications, it controls how the internal computation of the matrix multiplication is performed.

This does not change the precision of convolution operations. Other flags, like torch.backends.cudnn.allow_tf32, may control the precision of convolution operations.

This flag currently only affects one native device type: CUDA. If "high" or "medium" are set then the TensorFloat32 datatype will be used when computing float32 matrix multiplications, equivalent to setting torch.backends.cuda.matmul.allow_tf32 = True. When "highest" (the default) is set then the float32 datatype is used for internal computations, equivalent to setting torch.backends.cuda.matmul.allow_tf32 = False.

precision (str) – can be set to "highest" (default), "high", or "medium" (see above).